Inheritance

In C++

Introduction

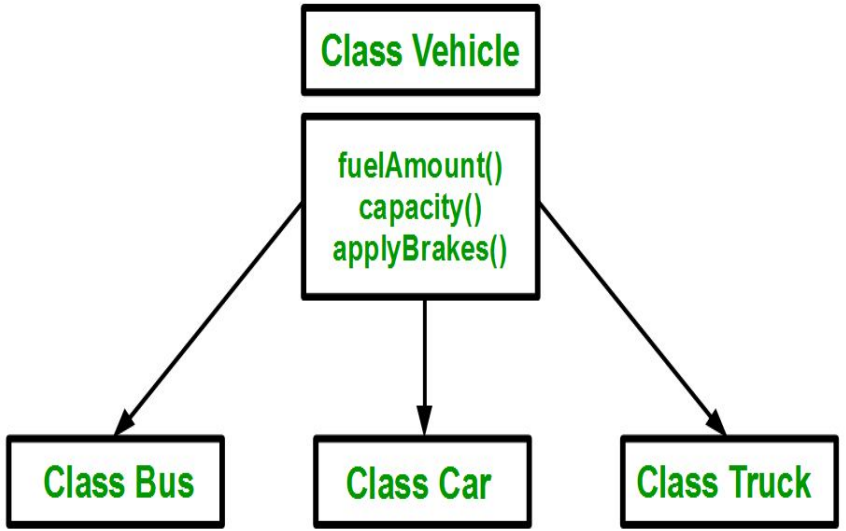
The capability of a class to derive properties and characteristics from another class is called **Inheritance**.

Inheritance is one of the most important feature of Object Oriented Programming.

Sub Class: The class that inherits properties from another class also called Derived Class.

Super Class: The class whose properties are inherited by Sub Class is called Base Class or Super class.

Using inheritance, we have to write the functions only one time instead of three times as we have inherited rest of the three classes from base class(Vehicle).



Implementing inheritance in C++: For creating a sub class which is inherited from the base class we have to follow the below syntax.

Syntax:

```
class subclass_name : access_mode base_class_name
{
 //body of subclass
};
```

Here, **subclass_name** is the name of the sub class, **access_mode** is the mode in which you want to inherit this sub class for example: public, private etc. and **base_class_name** is the name of the base class from which you want to inherit the sub class.

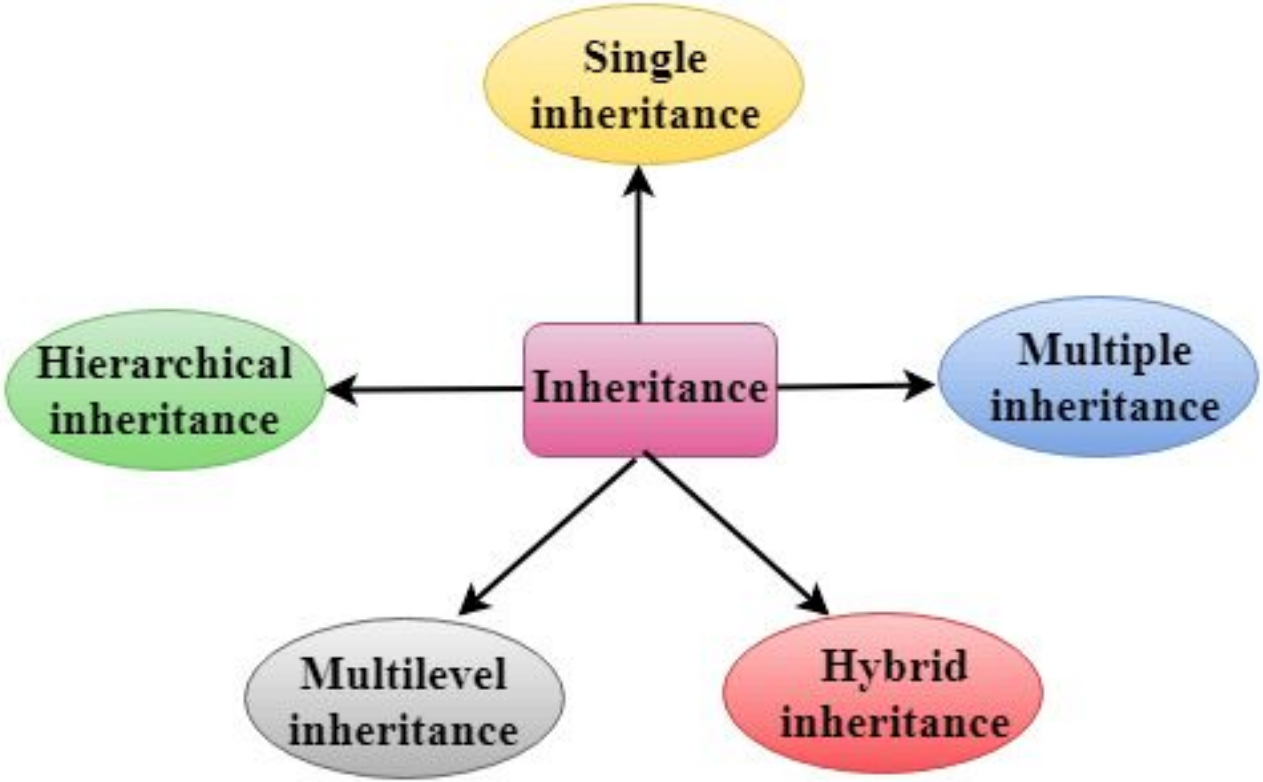
Modes of Inheritance

1. **Public mode**: If we derive a sub class from a public base class. Then the public member of the base class will become public in the derived class and protected members of the base class will become protected in derived class.
2. **Protected mode**: If we derive a sub class from a Protected base class. Then both public member and protected members of the base class will become protected in derived class.
3. **Private mode**: If we derive a sub class from a Private base class. Then both public member and protected members of the base class will become Private in derived class.

Note : The private members in the base class cannot be directly accessed in the derived class, while protected members can be directly accessed.

Base class member access specifier	Kind of Inheritance		
	public inheritance	protected inheritance	private inheritance
public	public in derived class Can be accessed directly by member functions, friend functions, and non-member functions	protected in derived class Can be accessed directly by member functions and friend functions	private in derived class Can be accessed directly by member functions and friend functions
protected	protected in derived class Can be accessed directly by member functions and friend functions	protected in derived class Can be accessed directly by member functions and friend functions	private in derived class Can be accessed directly by member functions and friend functions
private	Hidden in derived class Can be accessed by member functions and friend functions through public or protected member functions of the base class	Hidden in derived class Can be accessed by member functions and friend functions through public or protected member functions of the base class	Hidden in derived class Can be accessed by member functions and friend functions through public or protected member functions of the base class

Types of Inheritance

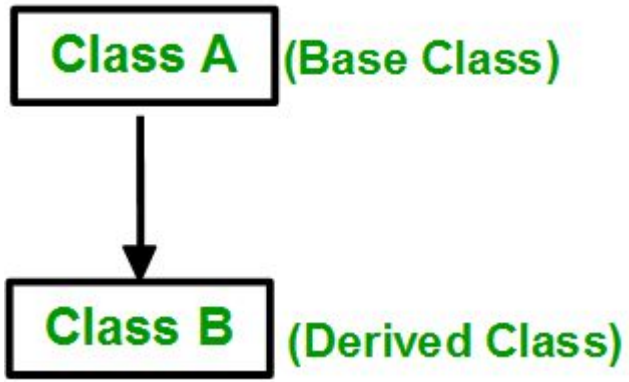


Single Inheritance

In single inheritance, a class is allowed to inherit from only one class. i.e. one sub class is inherited by one base class only.

Syntax:

```
class subclass_name : access_mode base_class
{
  //body of subclass
    };
```



Example:

```
#include <iostream>
using namespace std;
 class Account {
   public:
   float salary = 60000;
 };
   class Programmer: public Account {
   public:
   float bonus = 5000;
   };
int main(void) {
     Programmer p1;
     cout<<"Salary: "<<p1.salary<<endl;
     cout<<"Bonus: "<<p1.bonus<<endl;
    return 0;
}
```

Output:

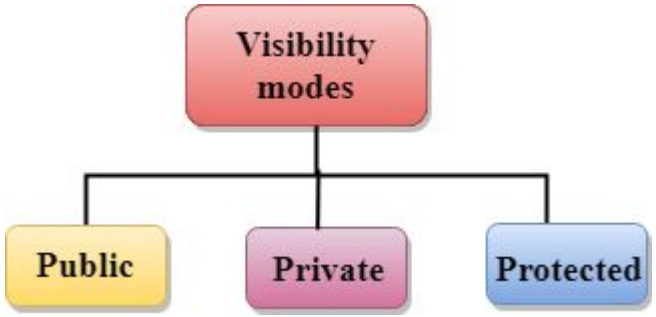
Salary: 60000

Bonus: 5000

How to make a Private Member Inheritable

The private member is not inheritable. If we modify the visibility mode by making it public, but this takes away the advantage of data hiding.

C++ introduces a third visibility modifier, i.e., **protected**. The member which is declared as protected will be accessible to all the member functions within the class as well as the class immediately derived from it.

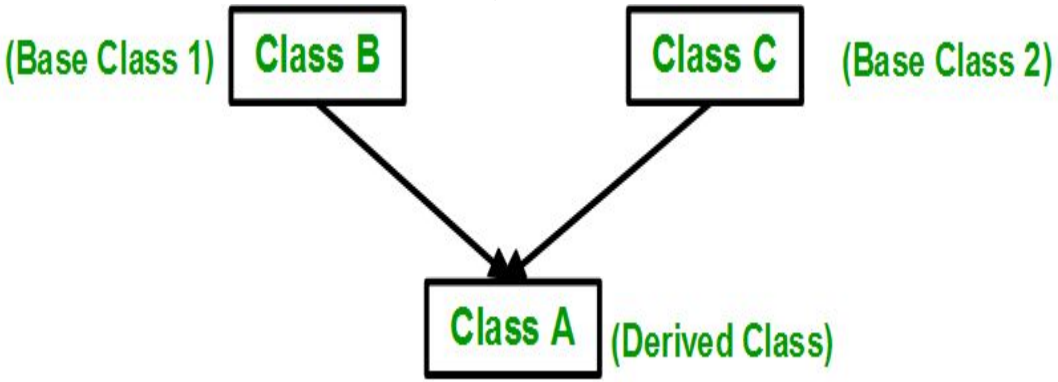


Multiple Inheritance

Multiple Inheritance: Multiple Inheritance is a feature of C++ where a class can inherit from more than one classes. i.e one **sub class** is inherited from more than one **base classes**. Here, the number of base classes will be separated by a comma (‘, ‘) and access mode for every base class must be specified.

Syntax:

```
class subclass_name : access_mode base_class1, access_mode base_class2, ....
{
    //body of subclass
};
```



Example:

```
#include <iostream>
using namespace std;
class A
{
   protected:
    int a;
   public:
   void get_a(int n)
   {
      a = n;
   }
};
class B
{
   protected:
   int b;
   public:
   void get_b(int n)
   {
      b = n;
   }
};
class C : public A,public B
{
  public:
   void display()
   {
    std::cout << "The value of a is : " <<a<<
std::endl;
        std::cout << "The value of b is : " <<b<<
std::endl;
        cout<<"Addition of a and b is : "<<a+b;
    }
};
int main()
{
   C c;
   c.get_a(10);
   c.get_b(20);
   c.display();
    return 0;
}
```

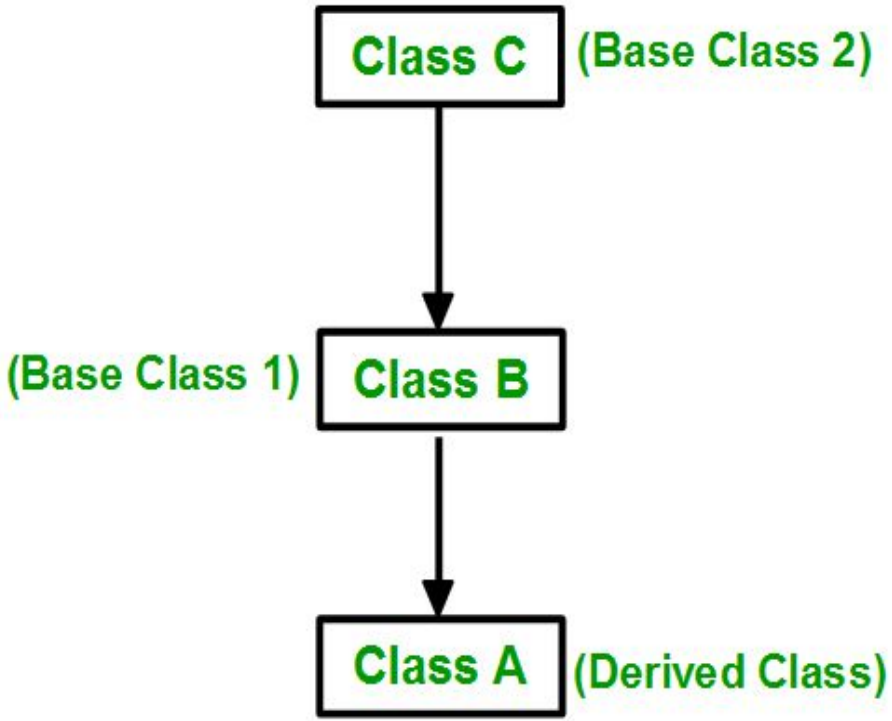
Output:

The value of a is : 10
The value of b is : 20

Addition of a and b is : 30

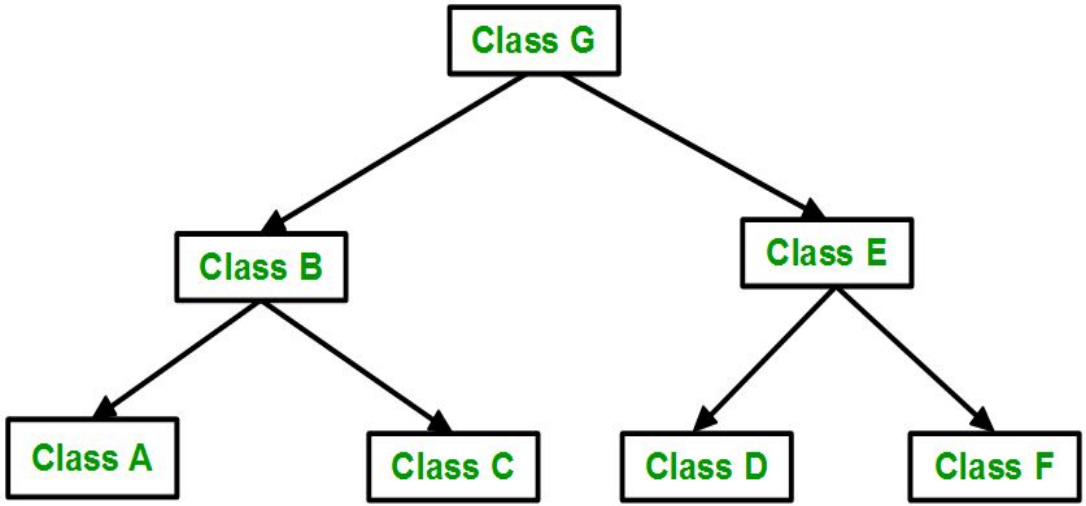
Multilevel Inheritance

Multilevel Inheritance: In this type of inheritance, a derived class is created from another derived class.



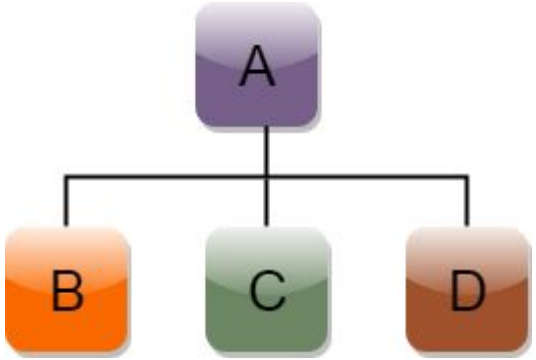
Hierarchical Inheritance

Hierarchical Inheritance: In this type of inheritance, more than one sub class is inherited from a single base class. i.e. more than one derived class is created from a single base class.



Example syntax:

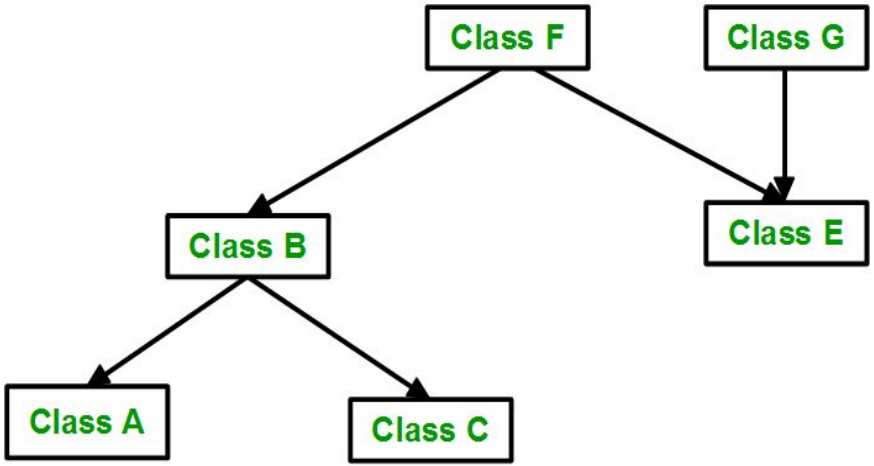
```
class A
{
   // body of the class A.  }
class B : public A
{
   // body of class B.  }
class C : public A
{
   // body of class C.  }
class D : public A
{
   // body of class D.  }
```



Hybrid Inheritance

Hybrid (Virtual) Inheritance: Hybrid Inheritance is implemented by combining more than one type of inheritance. For example: Combining Hierarchical inheritance and Multiple Inheritance.

Below image shows the combination of hierarchical and multiple inheritance:

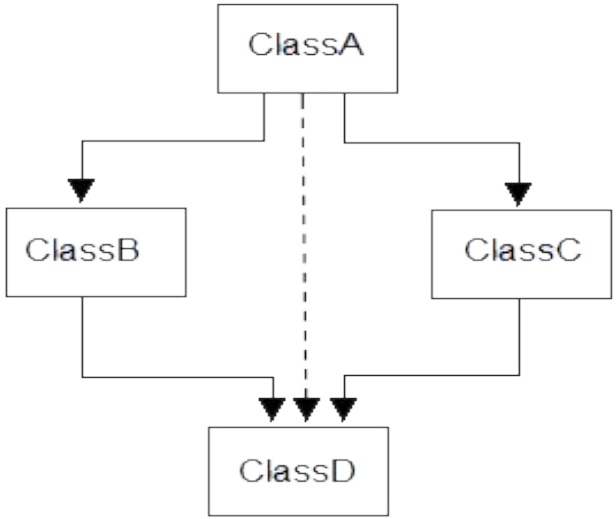


A special case of hybrid inheritance : Multipath inheritance

A derived class with two base classes and these two base classes have one common base class is called multipath inheritance. An ambiguity can arise in this type of inheritance.

There are 2 ways to avoid this ambiguity:

1. **Avoiding ambiguity using scope resolution operator** : Using scope resolution operator we can manually specify the path from which data member a will be accessed.
2. **Avoiding ambiguity using virtual base class**



Example:

```
class base1 {
  public:
     void someFunction( ) {....}
};
class base2 {
   void someFunction( ) {....}
};
class derived : public base1, public base2 {};
int main() {
   derived obj;
   obj.someFunction() // Error!
   }
```

This problem can be solved using the scope resolution function to specify which function to class either base1or base2

```
int main() {
   obj.base1::someFunction( );  // Function of base1 class is called
   obj.base2::someFunction();   // Function of base2 class is called.
   }
```

Virtual base classes

Virtual base classes are used in virtual inheritance in a way of preventing multiple “instances” of a given class appearing in an inheritance hierarchy when using multiple inheritances.

Need for Virtual Base Classes:

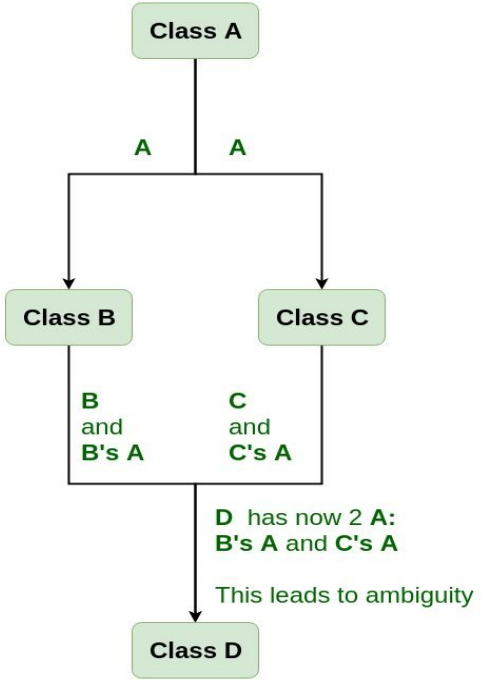
Consider the situation where we have one class **A** .

This class is **A** is inherited by two other classes **B** and **C**.

Both these class are inherited into another in a new class **D**

as shown in figure below.

As we can see from the figure that data members/function of class **A** are inheri through class **B** and second through class **C**. When any data / function membe object of class **D**, ambiguity arises as to which data/function member would be **B** or the other inherited through **C**. This confuses compiler and it displays error



Member classes: Nesting of classes

A nested class is a class that is declared in another class. The nested class is also a member variable of the enclosing class and has the same access rights as the other members.

Example:

```
#include<iostream>
using namespace std;
class A {
  public:
  class B {
    private:
    int num;
    public:
    void getdata(int n) {
      num = n;
    }
    void putdata() {
      cout<<"The number is "<<num;
    }
  };
};
int main() {
  cout<<"Nested classes in C++"<< endl;
  A :: B obj;
  obj.getdata(9);
  obj.putdata();
  return 0;
}
```

Output

Nested classes in C++
The number is 9

In the above program, class B is defined inside the class A so it is a nested class. The class B contains a private variable num and two public functions getdata() and putdata(). The function getdata() takes the data and the function putdata() displays the data.

Constructors in Derived Classes

1. The derived class need not have a constructor as long as the base class has a no-argument constructor.
2. However, if the base class has constructors with arguments (one or more), then it is mandatory for the derived class to have a constructor and pass the arguments to the base class constructor.
3. When an object of a derived class is created, the constructor of the base class is executed first and later the constructor of the derived class.

1. No constructors in the base class and derived class - Absence of constructors.

2. Constructor only in the base class

```
class A
{
public:
A() {
cout<<”No-argument constructor of the base class A is executed”;
}}
;
class B:public A
{
public:
};
void main()
{
B ob; //accesses base constructor
}
```

3. Constructor only in the derived class

```
class A
{
public:
};
class B:public A
{
public:
B(){
cout<<”No-argument constructor of the derived class B is executed”;
}
};
void main()
{
B ob; //accesses derived constructor
}
```

4. Constructors in both base and derived classes

```
class A
{
public:
A() {
cout<<”No-argument constructor of the base class A executed first”;
}}
;
class B:public A
{
public:
B()
{
cout<<”No-argument constructor of the derived class B is executed next”;
}
};
void main()
{
B ob; //accesses both constructor
}
```

Output:

No-argument constructor of the base class A executed first
No-argument constructor of the derived class B is executed next

5. Multiple constructors in base class and a single constructor in derived class

```
class A
{
public:
A()
{
cout<<"No argument constructor of base class A";
}
A(int a)
{
cout<<"One argument constructor of the base class A";
}
};
class B:public A
{
public:
B(int a)
{
cout<<"One argument constructor of the derived class B"
}}
;
void main()
{
B ob(3);
}
```

Output:

No argument constructor of base class A

One argument constructor of the derived class B

6. Constructor in base and derived classes without default constructor

```
class A
{
public:
A(int a)
{
cout<<”One argument constructor of the base class A”;
} };
class B:public A
{
public:
B(int a)
{
cout<<”One argument constructor of the derived class B”;
} };
```

```
void main()
{
B ob(3);
}
```

Output:

Error

7. Explicit invocation in the absence of default constructor

```
class A
{
public:
A(int a)
{
cout<<"One argument constructor of the base class A";
}};
class B:public A
{
public:
B(int a):A(a)
{
cout<<"One argument constructor of the derived class B";
}};
```

```
void main()
{
B ob(3);
}
```

Output:

One argument constructor of the base class A

One argument constructor of the derived class B

8. Constructor in a multiple inherited class with default invocation

```
class A1
{
public:
A1()
{
cout<<"No-argument constructor of the base class A1";
}};
class A2
{
public:
A2() {
cout<<"No argument constructor of the base class A2";
}};
class B:public A2, public A1
{
public:
B()
{
cout<<"No argument constructor of the derived class B
}};
void main()
{
B ob;
}
```

Output:

No argument constructor of the base **class A2**
No-argument constructor of the base class A1
No argument constructor of the derived class B

9. Constructor in a multiple inherited class with explicit invocation

```
class A1
{
public:
A1()
{
cout<<"No-argument constructor of the base class A1";
}};
class A2
{
public:
A2() {
cout<<"No argument constructor of the base class A2";
}};
class B:public
A2, public A1
{ public:
B():A2(),A1()
{
cout<<"No argument constructor of the derived class B
}};
void main()
{
B ob;
}
```

Output:

No argument constructor of the base class A2
No-argument constructor of the base class A1
No argument constructor of the derived class B

10. Constructor in multilevel inheritance

```
class A
{
public:
A()
{
cout<<”No-argument constructor of the base class A”;
}}
;
class B:public A
{
public:
B() {
cout<<”No argument constructor of the base class B”;
}};
class C:public B
{
public:
C()
{
cout<<”No argument constructor of the derived class
}}
;
void main()
{
C ob;
}
```

Output:

No argument constructor of the base class A

No-argument constructor of the base class B

No argument constructor of the derived class C

Destructors in derived classes

1. Unlike constructors, destructors in the class hierarchy (parent and child class) are invoked in the reverse order of the constructor invocation.

1. The destructor of that class whose constructor was executed last, while building object of the derived class, will be executed first whenever the object goes out of scope.

Example:

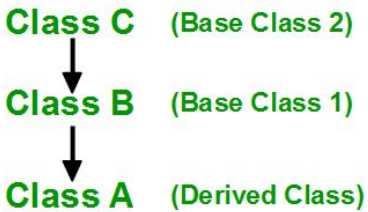
```
class B1
{
public:
B1() {
cout<<”No argument constructor of the base class B1”;
}
~B1()
{
cout<<”Destructor in the base class B1”;
}};
class B2
{
public:
B2()
{
cout<<”No argument constructor of the base class B2”; }
~B2()
{
cout<<”Destructor in the base class B2”;
}}
;
class D:public B1,public B2
{
public:
D()
{
cout<<”No argument constructor of the derived
class D”;
}
```

```
~D()
{
cout<<”Destructor in the derived class D”;
}}
;
void main()
{
D ob;
}
```

Output:

```
No argument constructor of the base class B1
No argument constructor of the base class B2
No argument constructor of the derived class D
Destructor in the derived class D
Destructor in the base class B2
Destructor in the base class B1
```

Order of Inheritance



Order of Constructor Call

1. C() (Class C's Constructor)
2. B() (Class B's Constructor)
3. A() (Class A's Constructor)

Order of Destructor Call

1. ~A() (Class A's Destructor)
2. ~B() (Class B's Destructor)
3. ~C() (Class C's Destructor)